

Operating Systems Interview Guide

1. Overview

Operating Systems, or OS, manage the hardware and provide services to programs. For interviews, OS questions usually test whether you understand how programs actually run: how CPU time is shared, how memory is managed, how processes communicate, and how the system protects itself.

Definition

An operating system is system software that manages resources such as CPU, memory, files, I/O devices, and processes, while providing abstractions like processes, threads, virtual memory, files, and sockets.

Why It Matters

Most backend, systems, database, browser, and application performance issues eventually touch OS concepts:

- Slow program? Could be context switching, I/O blocking, paging, or scheduling.
- Crash? Could be invalid memory access, protection fault, or resource exhaustion.
- Server hangs? Could be deadlock, race condition, or thread starvation.
- High memory usage? Could involve virtual memory, page faults, or process isolation.

Where It Is Used In Real Systems

- Backend servers use processes, threads, sockets, mutexes, and kernel syscalls.
- Databases use locks, semaphores, memory-mapped files, paging, and buffer pools.
- Browsers use multiple processes for tabs, sandboxing, and memory isolation.
- Mobile apps use user/kernel mode transitions for file, network, camera, and UI operations.
- Containers use OS isolation features such as namespaces, cgroups, virtual memory, and process control.

Why Interviewers Ask About It

Interviewers ask OS questions to check:

- Whether you understand what happens below application code.
- Whether you can reason about concurrency bugs.
- Whether you know performance costs such as context switching and page faults.
- Whether you can explain practical flows like running a command in a terminal.
- Whether you can compare concepts clearly under pressure.

2. Core Idea

The core idea of OS is controlled sharing.

Many programs want to use the same CPU, memory, disk, and devices. The OS makes this sharing safe, fair, and efficient.

Intuition

Think of the OS as a manager of a large office:

- CPU is the meeting room.
- Processes are teams.
- Threads are people within a team.
- Memory is office space.
- Files are shared documents.
- Kernel mode is the manager-only control room.
- User mode is the normal employee area.
- Locks are rules for using shared equipment.

Without the OS, programs could overwrite each other's memory, monopolize the CPU, corrupt files, or directly misuse devices.

Small Example

Suppose you open a browser, play music, and compile code at the same time.

The OS:

1. Gives each program a process.
2. Schedules CPU time among them.
3. Gives each process virtual memory.
4. Switches between processes using context switches.
5. Handles disk, keyboard, display, and network I/O through kernel services.
6. Prevents one program from directly corrupting another program's memory.

Step-By-Step: How OS Makes Programs Run

1. Program is stored as an executable file on disk.
2. User starts it from terminal, GUI, or another program.
3. OS creates a process.
4. OS loads code and data into the process's virtual address space.
5. OS creates at least one thread.
6. Scheduler gives the thread CPU time.
7. Program executes in user mode.
8. For privileged operations like file or network access, program makes system calls.
9. Kernel handles those requests in kernel mode.
10. OS may switch CPU to another process or thread whenever needed.

3. Important Subtopics

3.1 Process

What It Means

A process is a running instance of a program. It has its own address space, resources, file descriptors, registers, stack, heap, and process control block.

Why It Matters

Processes provide isolation. If one process crashes, it usually does not directly corrupt another process.

Example

Opening two Chrome tabs may create multiple processes. Running `python app.py` creates a Python process.

Common Interview Angle

Interviewers often ask:

- What is the difference between a program and a process?
- Why are processes heavier than threads?
- What does the OS store for each process?

3.2 Thread

What It Means

A thread is the smallest unit of CPU execution inside a process. Threads of the same process share code, heap, and open files, but each thread has its own stack, registers, and program counter.

Why It Matters

Threads allow concurrent execution within the same process. They are useful for servers, UI apps, parallel computation, and background tasks.

Example

A web server may create one thread per request or use a thread pool to handle many requests.

Common Interview Angle

Interviewers often ask:

- Why are threads faster to create than processes?
- What resources are shared between threads?
- Why can threads cause race conditions?

3.3 Process Vs Thread

Feature	Process	Thread
Meaning	Running program instance	Execution unit inside a process
Memory	Separate address space	Shares process address space
Isolation	Strong	Weak between threads of same process
Communication	Slower, uses IPC	Faster, shared memory
Creation cost	Higher	Lower

Feature	Process	Thread
Context switch cost	Higher	Usually lower
Crash impact	Usually isolated	Can crash entire process
Example	Browser process	Worker thread in browser process

3.4 User Mode Vs Kernel Mode

What It Means

Modern CPUs support privilege levels.

- User mode: normal application code runs here with limited privileges.
- Kernel mode: OS kernel runs here with full hardware access.

Why It Matters

This separation protects the system. Applications cannot directly access hardware, modify page tables, disable interrupts, or read another process's memory.

Example

When a program calls `read()` to read a file:

1. Program runs in user mode.
2. It executes a system call.
3. CPU switches to kernel mode.
4. Kernel validates permissions and performs file I/O.
5. CPU returns to user mode.

Common Interview Angle

Interviewers may ask:

- Why do we need user mode and kernel mode?
- What happens during a system call?
- Is switching to kernel mode the same as a context switch?

Important: A mode switch is not always a full process context switch.

3.5 Context Switch

What It Means

A context switch happens when the CPU stops running one thread or process and starts running another.

The OS saves the current execution context and restores another one.

Why It Matters

Context switching allows multitasking, but it has overhead. Too many context switches can reduce performance.

What Gets Saved

Typically:

- Program counter
- CPU registers
- Stack pointer
- Process/thread state
- Memory management information
- Scheduling information

Step-By-Step Context Switch

```
Running Thread A
  |
  v
Timer interrupt / I/O wait / higher priority task
  |
  v
CPU enters kernel mode
  |
  v
OS saves context of Thread A
  |
  v
Scheduler chooses Thread B
  |
  v
OS restores context of Thread B
  |
  v
CPU resumes Thread B
```

Common Interview Angle

Interviewers ask:

- What causes a context switch?
- Why is context switching expensive?
- Difference between process context switch and thread context switch?

3.6 Mutex

What It Means

A mutex, or mutual exclusion lock, allows only one thread to enter a critical section at a time.

Why It Matters

It prevents race conditions when multiple threads access shared mutable data.

Example

```
lock(mutex)
balance = balance + 100
unlock(mutex)
```

Only one thread can update **balance** at once.

Common Interview Angle

Interviewers ask:

- Why do we need mutexes?
- What happens if a thread forgets to unlock?
- Can mutexes cause deadlock?

3.7 Semaphore

What It Means

A semaphore is a synchronization primitive that maintains a counter.

- Counting semaphore: allows up to N threads to access a resource.
- Binary semaphore: counter is 0 or 1, similar to a lock but not always ownership-based.

Why It Matters

Semaphores control access to a limited number of resources, such as database connections or buffer slots.

Example

If a connection pool has 10 connections, a semaphore initialized to 10 allows only 10 threads to acquire connections at once.

Common Interview Angle

Interviewers ask:

- Difference between mutex and semaphore?
- What are **wait()** and **signal()**?
- Can a semaphore be used for producer-consumer problems?

3.8 Race Condition

What It Means

A race condition occurs when program correctness depends on the unpredictable timing or ordering of threads.

Why It Matters

Race conditions cause inconsistent, hard-to-debug failures.

Example

Two threads increment the same variable:

```
counter = counter + 1
```

This may internally be:

```
read counter
add 1
write counter
```

If two threads interleave incorrectly, one update may be lost.

Common Interview Angle

Interviewers expect you to explain that simple-looking operations may not be atomic.

3.9 Deadlock

What It Means

A deadlock occurs when two or more processes or threads wait forever for resources held by each other.

Four Necessary Conditions

Deadlock can happen only if all four conditions hold:

Condition	Meaning
Mutual exclusion	Resource cannot be shared simultaneously
Hold and wait	A process holds one resource while waiting for another
No preemption	Resource cannot be forcibly taken away
Circular wait	Processes form a cycle of waiting

Example

```
Thread A holds Lock 1 and waits for Lock 2
Thread B holds Lock 2 and waits for Lock 1
```

Prevention

Break at least one of the four conditions:

- Avoid hold and wait by acquiring all resources together.
- Allow preemption where possible.
- Enforce a global lock ordering to prevent circular wait.
- Reduce mutual exclusion when resources can be shared safely.
- Use timeouts and retry logic.

Common Interview Angle

Interviewers often ask for the four conditions and one practical prevention strategy.

3.10 Paging

What It Means

Paging divides virtual memory into fixed-size pages and physical memory into fixed-size frames.

The OS maps virtual pages to physical frames using page tables.

Why It Matters

Paging enables virtual memory, isolation, efficient allocation, and demand loading.

Example

Virtual page 5 of a process may be mapped to physical frame 20 in RAM.

Common Interview Angle

Interviewers ask:

- What is a page table?
- What is a TLB?
- What happens during a page fault?

3.11 Segmentation

What It Means

Segmentation divides memory into logical variable-sized segments such as code, stack, heap, and data.

Why It Matters

Segmentation matches the programmer's logical view of memory, but variable-sized allocation can cause external fragmentation.

Example

A process may have:

```
Code segment
Data segment
Heap segment
Stack segment
```

Common Interview Angle

Interviewers ask paging vs segmentation and fragmentation differences.

3.12 Virtual Memory

What It Means

Virtual memory gives each process the illusion of having a large, private, continuous memory space.

The OS and hardware translate virtual addresses to physical addresses.

Why It Matters

Virtual memory provides:

- Process isolation
- Memory protection
- Efficient memory usage
- Demand paging
- Ability to run programs larger than physical RAM

Example

Two processes may both use virtual address `0x400000`, but those addresses map to different physical frames.

Common Interview Angle

Interviewers ask:

- Why do we need virtual memory?
- Can virtual memory be larger than RAM?
- What is the role of page tables?

3.13 Page Fault

What It Means

A page fault occurs when a process accesses a virtual page that is not currently mapped in physical memory or violates access permissions.

Why It Matters

Page faults are part of demand paging, but excessive page faults cause severe slowdown.

Step-By-Step Page Fault Handling

```

Process accesses virtual address
  |
  v
MMU checks page table
  |
  v
Page not present or permission violation
  |
  v
CPU traps into kernel
  |
  v
OS checks if access is valid
  |
  +-- Invalid access --> terminate process / segmentation fault
  |
  +-- Valid access
        |
        v
        Find free frame or evict another page
        |
        v
        Load required page from disk if needed
        |
        v
        Update page table and TLB
        |
        v
        Restart faulting instruction

```

Common Interview Angle

Interviewers expect you to distinguish a valid page fault from an invalid memory access.

3.14 Running A Program Or Typing A Command In Terminal

What It Means

When you type a command, the shell parses it and asks the OS to create and run a process.

Why It Matters

This combines many OS concepts: shell, process creation, executable loading, virtual memory, syscalls, scheduling, I/O, and termination.

Example

Command:

```
ls -l
```

Typical flow:

1. Terminal receives keyboard input.
2. Shell reads the command.
3. Shell parses command and arguments.
4. Shell checks if it is a built-in command.
5. If not built-in, shell searches executable in `PATH`.
6. Shell creates a child process.
7. Child process loads the executable.
8. OS sets up virtual memory, stack, heap, file descriptors, and environment variables.
9. Scheduler runs the process.
10. Program uses system calls to read directories and write output.
11. Output appears in terminal.
12. Program exits and returns an exit status.
13. Shell collects status and shows prompt again.

Common Interview Angle

Interviewers may ask:

- What happens when you run `./a.out`?
- What is the role of shell?
- What is the difference between shell built-in and external command?
- What are `fork()` and `exec()` in Unix-like systems?

4. Real-World Example

Backend Server Handling Requests

Consider a backend server handling many client requests.

1. The server process starts when deployed.
2. It creates worker threads or uses an event loop.
3. Each request may access shared resources like cache, logs, database connections, or counters.
4. Mutexes protect shared data.
5. Semaphores limit resources like database connections.
6. OS scheduler switches between threads.
7. If a thread performs disk or network I/O, it may block and the CPU runs another thread.

8. Virtual memory isolates the server process from other processes.
9. If memory pressure is high, pages may be evicted and later page faults may occur.
10. If locks are acquired in inconsistent order, deadlock can happen.

This is why OS concepts are directly useful in SDE interviews and production debugging.

5. Diagrams / Mental Models

Process And Threads

```

Process
|
+-- Address space
|   +-- Code
|   +-- Heap
|   +-- Global data
|
+-- Open files
+-- File descriptors
+-- Thread 1
|   +-- Stack
|   +-- Registers
|
+-- Thread 2
    +-- Stack
    +-- Registers
  
```

User Mode And Kernel Mode

```

Application code
|
| system call: read(), write(), open(), socket()
v
Kernel mode
|
| validate request, access hardware/resource
v
Return to user mode
  
```

Mutex Mental Model

Shared resource: account balance

```

Thread A ---- acquire lock ---- update ---- release lock
Thread B ---- waits ----- update ---- release lock
  
```

Deadlock Cycle

```

Thread A holds Lock 1
Thread A waits for Lock 2

Thread B holds Lock 2
Thread B waits for Lock 1

A <---- waits ---- B
B <---- waits ---- A

```

Virtual Memory Translation

```

Virtual address
  |
  v
MMU checks TLB
  |
  +-- TLB hit --> physical address
  |
  +-- TLB miss
      |
      v
      Page table lookup
      |
      +-- present --> physical address
      |
      +-- not present --> page fault

```

Terminal Command Execution

```

User types command
  |
  v
Shell parses command
  |
  v
Built-in?
  | yes
  v
Run inside shell

  | no
  v
Find executable in PATH
  |
  v
Create child process

```

```
    |  
    v  
Load program  
    |  
    v  
Run and produce output  
    |  
    v  
Exit status returned to shell
```

6. Common Interview Questions

1. What is the difference between a process and a thread?

Answer

A process is a running program with its own address space and resources. A thread is an execution unit inside a process. Threads in the same process share memory and resources but have separate stacks and registers.

Key Points Interviewer Expects

- Process has separate address space.
- Threads share process memory.
- Process context switching is generally heavier.
- Threads are useful for concurrency but need synchronization.

Common Mistakes

- Saying thread and process are the same.
- Forgetting that each thread has its own stack.
- Saying threads are always faster without mentioning synchronization risks.

2. Why are threads called lightweight processes?

Answer

Threads are called lightweight because creating and switching between threads usually requires less overhead than processes. Threads share the same address space, code, heap, and open files of their process.

Key Points Interviewer Expects

- Shared address space reduces overhead.
- Thread creation is cheaper than process creation.
- Communication between threads is easier through shared memory.

Common Mistakes

- Assuming threads are always cheap in every language or runtime.
- Ignoring locking and race condition costs.

3. What is user mode and kernel mode?

Answer

User mode is where normal applications run with limited privileges. Kernel mode is where the OS runs with full hardware access. Applications request privileged services through system calls.

Key Points Interviewer Expects

- Protection and security.
- System calls switch from user mode to kernel mode.
- Kernel can access hardware and manage memory.

Common Mistakes

- Saying every function call enters kernel mode.
- Confusing mode switch with process context switch.

4. What happens during a context switch?

Answer

The OS saves the CPU state of the currently running process or thread, chooses another runnable process or thread using the scheduler, restores its saved state, and resumes execution.

Key Points Interviewer Expects

- Save registers, program counter, stack pointer.
- Scheduler selects next task.
- Restore context of next task.
- It has overhead.

Common Mistakes

- Saying context switch is free.
- Forgetting that interrupts or blocking I/O can trigger it.
- Not distinguishing thread switch and process switch.

5. What is a race condition?

Answer

A race condition occurs when multiple threads access shared data and the result depends on the timing or interleaving of operations.

Key Points Interviewer Expects

- Shared mutable state.
- Non-atomic operations.

- Incorrect interleaving.
- Use locks, atomics, or proper synchronization.

Common Mistakes

- Thinking race conditions happen only in multi-core systems.
- Believing `x++` is always atomic.

6. What is a mutex?

Answer

A mutex is a lock used to ensure mutual exclusion. Only one thread can hold the mutex and enter the protected critical section at a time.

Key Points Interviewer Expects

- Protects critical section.
- Usually has ownership: the thread that locks should unlock.
- Prevents races but can cause deadlocks if misused.

Common Mistakes

- Using mutex for resource counting.
- Forgetting to unlock on error paths.

7. What is a semaphore?

Answer

A semaphore is a synchronization primitive with a counter. `wait()` decreases the counter or blocks if it is zero. `signal()` increases the counter and may wake a waiting thread.

Key Points Interviewer Expects

- Counter-based synchronization.
- Binary and counting semaphores.
- Useful for producer-consumer and limited resource pools.

Common Mistakes

- Saying semaphore and mutex are always identical.
- Ignoring that semaphores may not have strict ownership.

8. What are the four conditions for deadlock?

Answer

The four necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait.

Key Points Interviewer Expects

- Explain all four conditions.
- Deadlock requires all four.
- Prevention means breaking at least one condition.

Common Mistakes

- Listing only circular wait.
- Confusing deadlock with starvation.

9. How can deadlock be prevented?**Answer**

Deadlock can be prevented by breaking one of the four necessary conditions. A practical method is enforcing a global lock ordering so circular wait cannot happen.

Key Points Interviewer Expects

- Lock ordering.
- Acquire all resources together.
- Timeouts and retries.
- Resource preemption where possible.

Common Mistakes

- Saying "use mutex" to prevent deadlock.
- Ignoring that mutex misuse can cause deadlock.

10. What is virtual memory?**Answer**

Virtual memory is an abstraction that gives each process its own private address space. The OS and hardware map virtual addresses to physical memory using page tables.

Key Points Interviewer Expects

- Isolation.
- Protection.
- Address translation.
- Demand paging.

Common Mistakes

- Saying virtual memory is only disk space.
- Forgetting the role of page tables and MMU.

11. What is paging?

Answer

Paging divides virtual memory into fixed-size pages and physical memory into frames. Page tables map pages to frames.

Key Points Interviewer Expects

- Fixed-size blocks.
- Avoids external fragmentation.
- Can cause internal fragmentation.
- Supports virtual memory.

Common Mistakes

- Confusing pages with segments.
- Saying paging always means swapping to disk.

12. What is segmentation?

Answer

Segmentation divides memory into logical variable-sized segments such as code, data, heap, and stack.

Key Points Interviewer Expects

- Logical memory division.
- Variable-sized segments.
- Can suffer from external fragmentation.

Common Mistakes

- Saying segmentation uses fixed-size blocks.
- Ignoring external fragmentation.

13. What happens during a page fault?

Answer

When a process accesses a page that is not present or violates permissions, the CPU traps into the kernel. The OS checks whether the access is valid. If valid, it loads or maps the page, updates page tables, and restarts the instruction. If invalid, it terminates the process or raises an error.

Key Points Interviewer Expects

- Trap to kernel.
- Validate access.
- Load page or allocate frame.

- Update page table/TLB.
- Resume or terminate.

Common Mistakes

- Saying every page fault is an error.
- Forgetting invalid page faults cause segmentation faults or access violations.

14. What happens when you type a command in terminal?

Answer

The shell reads and parses the command. If it is built-in, the shell executes it directly. Otherwise, it finds the executable, creates a child process, loads the program, sets up arguments/environment/file descriptors, runs it, waits for completion if foreground, and displays the output and prompt.

Key Points Interviewer Expects

- Shell parsing.
- Built-in vs external command.
- Process creation.
- Program loading.
- System calls for I/O.
- Exit status.

Common Mistakes

- Ignoring the shell.
- Saying terminal itself executes every command.
- Forgetting environment variables and `PATH`.

7. Deep-Dive Questions

1. Is a system call the same as a context switch?

No. A system call usually causes a switch from user mode to kernel mode, but the same process may continue running. A context switch changes the currently running thread or process. A system call may lead to a context switch if the process blocks, for example while waiting for disk I/O.

2. Why is process context switching usually more expensive than thread context switching?

Process context switching may require changing the virtual address space, page table base register, and possibly flushing or affecting TLB entries. Threads in the same process share address space, so switching between them can be cheaper. However, exact cost depends on OS, CPU, and workload.

3. Can a page fault happen even when there is free RAM?

Yes. A page fault simply means the requested virtual page is not currently mapped as needed. The OS may allocate a free frame and load or zero-fill the page. Free RAM avoids eviction but does not eliminate all page faults.

4. What is the difference between deadlock and starvation?

Deadlock means a set of threads are permanently waiting for each other in a cycle. Starvation means a thread waits for a long time because it is repeatedly denied CPU or resources, even though progress is possible for others.

5. What happens if two threads access different variables on the same cache line?

This can cause false sharing. Even though variables are logically independent, CPU cache coherence works at cache-line granularity. Frequent writes by different cores can invalidate each other's cache lines and reduce performance.

8. Comparison Tables

Process Vs Thread

Aspect	Process	Thread
Basic unit	Resource ownership and execution container	CPU execution unit
Address space	Separate	Shared within process
Communication	IPC needed	Shared memory possible
Safety	More isolated	Less isolated
Creation	Slower	Faster
Switching	More expensive	Usually cheaper
Crash	Usually affects one process	May affect entire process
Best used for	Isolation, independent programs	Concurrency inside same app

User Mode Vs Kernel Mode

Aspect	User Mode	Kernel Mode
Runs	Applications	OS kernel
Privilege	Limited	Full
Hardware access	Not direct	Direct or controlled
Memory access	Own process memory	Kernel memory and controlled process memory
Failure impact	Usually process crash	Can crash entire system
Example	Normal C++/Java/Python code	Scheduler, memory manager, device driver

Mutex Vs Semaphore

Aspect	Mutex	Semaphore
Purpose	Mutual exclusion	Resource counting/signaling

Aspect	Mutex	Semaphore
Counter	Usually locked/unlocked	Integer counter
Ownership	Usually owned by locking thread	May not have strict ownership
Access allowed	One thread	Up to N threads
Used for	Critical sections	Resource pools, producer-consumer
Common bug	Deadlock if not unlocked	Wrong signal/wait count

Race Condition Vs Deadlock

Aspect	Race Condition	Deadlock
Meaning	Result depends on timing	Threads wait forever
Cause	Unsynchronized shared data	Circular resource waiting
Symptom	Wrong/inconsistent output	Program hangs
Prevention	Locks, atomics, immutability	Lock ordering, timeouts, avoid hold-and-wait
Example	Lost counter update	Lock A waits for B, B waits for A

Paging Vs Segmentation

Aspect	Paging	Segmentation
Division	Fixed-size pages	Variable-size logical segments
View	OS/hardware memory management	Programmer/logical view
Fragmentation	Internal fragmentation possible	External fragmentation possible
Address	Page number + offset	Segment number + offset
Allocation	Easier	More complex
Example	4 KB pages	Code, stack, heap, data

Page Fault Vs Segmentation Fault

Aspect	Page Fault	Segmentation Fault
Meaning	Page not present or permission issue	Invalid memory access error
Always fatal?	No	Usually fatal to process
Handled by	OS kernel	OS reports error to process
Example	Demand loading a page	Dereferencing invalid pointer
Interview trap	Page fault can be normal	Segmentation fault is an invalid access

Program Vs Process

Aspect	Program	Process
Meaning	Passive executable file	Running instance
Location	Disk/storage	Memory and OS process table
State	No runtime state	Has runtime state
Example	<code>chrome.exe</code> file	Running Chrome process

9. Common Mistakes

- Saying a program and a process are the same.
- Saying threads do not have their own stack.
- Assuming threads are always better than processes.
- Confusing concurrency with parallelism.
- Saying a system call is always a context switch.
- Saying page fault always means program crash.
- Confusing page fault with segmentation fault.
- Saying mutex and semaphore are identical.
- Forgetting the four deadlock conditions.
- Thinking deadlock and starvation are the same.
- Saying paging has external fragmentation.
- Saying segmentation has fixed-size blocks.
- Ignoring `PATH`, shell built-ins, arguments, and environment variables when explaining terminal command execution.
- Forgetting that simple operations like increment are often not atomic.

10. Edge Cases / Special Cases

Mode Switch Without Context Switch

A system call switches from user mode to kernel mode, but the same process can continue after the syscall. This is not necessarily a full context switch.

Context Switch Within Same Process

Switching between two threads of the same process is still a context switch, but it may be cheaper than switching between two different processes.

Valid Page Fault

A page fault may be valid and expected, such as when a process accesses a page that should be demand-loaded from disk or zero-filled by the OS.

Invalid Page Fault

If a process accesses memory outside its valid address space or violates permissions, the OS may terminate it with a segmentation fault or access violation.

Binary Semaphore Is Not Always A Mutex

A binary semaphore has values 0 and 1, but a mutex usually has ownership rules. In many systems, only the thread that locked a mutex should unlock it.

Deadlock With One Thread

A single thread can deadlock itself if it tries to acquire a non-recursive mutex it already holds.

Shell Built-Ins

Commands like `cd` are usually shell built-ins because they must modify the shell's own state. If `cd` ran only in a child process, the parent shell's directory would not change.

Copy-On-Write In Process Creation

In Unix-like systems, `fork()` often uses copy-on-write. Parent and child initially share physical pages marked read-only. Actual copying happens only when one writes.

11. How To Explain In Interview

Operating systems manage resources like CPU, memory, files, and devices. A process is a running program with its own address space, while a thread is an execution path inside a process that shares memory with other threads. User mode protects the system by limiting application privileges, and kernel mode is used by the OS for privileged operations through system calls. Context switching lets the CPU move between tasks by saving and restoring execution state. For synchronization, mutexes protect critical sections, semaphores manage limited resources, and incorrect synchronization can cause race conditions or deadlocks. Memory is managed using virtual memory, paging, and page tables, and a page fault occurs when a needed page is missing or access is invalid.

12. Quick Revision Notes

Key Definitions

- Process: running instance of a program.
- Thread: execution unit inside a process.
- User mode: restricted mode for applications.
- Kernel mode: privileged mode for OS code.
- Context switch: saving one task's state and restoring another's.
- Mutex: lock for one-at-a-time critical section access.
- Semaphore: counter-based synchronization primitive.
- Race condition: output depends on timing of threads.
- Deadlock: threads wait forever due to circular dependency.
- Paging: fixed-size memory management.
- Segmentation: logical variable-sized memory division.
- Virtual memory: private address space abstraction.
- Page fault: trap caused by missing page or invalid access.

Important Points

- Process isolation improves safety.

- Threads are cheaper but share memory, so synchronization matters.
- System calls move execution into kernel mode.
- Context switching is necessary but costly.
- Deadlock requires four conditions.
- Paging avoids external fragmentation.
- Segmentation can suffer external fragmentation.
- Page faults can be normal.

Common Comparisons

- Process vs thread
- User mode vs kernel mode
- Mutex vs semaphore
- Race condition vs deadlock
- Paging vs segmentation
- Page fault vs segmentation fault

Must-Remember Facts

- Threads share heap but have separate stacks.
- A process has its own virtual address space.
- A system call is not always a context switch.
- A page fault is not always fatal.
- Deadlock prevention means breaking at least one necessary condition.
- `cd` is usually a shell built-in.

Interview Traps

- Do not say virtual memory is just swap space.
- Do not say mutex and semaphore are exactly the same.
- Do not ignore shell involvement when explaining terminal commands.
- Do not say paging has external fragmentation.
- Do not confuse deadlock with starvation.

13. Practice Tasks

Task 1: Trace A Context Switch

Draw what happens when Thread A is running, a timer interrupt occurs, and the scheduler chooses Thread B.

Focus on:

- Saved registers
- Program counter
- Stack pointer
- Scheduler decision
- Restored context

Task 2: Simulate A Race Condition

Write a small program in C++, Java, or Python where two threads increment a shared counter many times.

Then:

- Run without a lock.
- Run with a lock.
- Compare results.

Task 3: Producer-Consumer With Semaphore

Implement a bounded buffer using:

- One mutex
- One **empty** semaphore
- One **full** semaphore

Explain why each one is needed.

Task 4: Deadlock Example

Create two locks:

```
Thread A: lock L1, then L2
Thread B: lock L2, then L1
```

Explain why this can deadlock and fix it using global lock ordering.

Task 5: Explain Terminal Command Execution

Pick this command:

```
cat input.txt | grep hello > output.txt
```

Explain:

- Shell parsing
- Pipes
- Redirection
- Process creation
- File descriptors
- Execution flow

Task 6: Page Fault Walkthrough

Draw the flow when a process accesses a valid page that is currently not in RAM.

Mention:

- MMU
- Page table
- Trap to kernel
- Disk or zero-fill
- Page table update
- Instruction restart

Task 7: Compare Paging And Segmentation

Create a table comparing:

- Block size
- Fragmentation
- Address format
- Programmer view
- OS usage

14. Final Cheat Sheet

Core Definition

An operating system manages CPU, memory, files, I/O, processes, and security so multiple programs can run safely and efficiently on the same machine.

Why It Matters

OS concepts explain real performance, memory, and concurrency behavior in applications, servers, databases, browsers, and distributed systems.

Most Asked Questions

- Process vs thread
- User mode vs kernel mode
- What happens during a context switch?
- Mutex vs semaphore
- What is a race condition?
- What are deadlock conditions?
- Paging vs segmentation
- What is virtual memory?
- What happens during a page fault?
- What happens when you run a command in terminal?

Common Comparisons

Comparison	One-Line Difference
Process vs Thread	Process has separate memory; threads share process memory
User vs Kernel Mode	User mode is restricted; kernel mode is privileged

Comparison	One-Line Difference
Mutex vs Semaphore	Mutex protects one critical section; semaphore counts available resources
Race vs Deadlock	Race gives wrong result; deadlock causes waiting forever
Paging vs Segmentation	Paging is fixed-size; segmentation is logical and variable-size
Page Fault vs Segmentation Fault	Page fault can be valid; segmentation fault is invalid access

One-Line Interview Answer

OS manages resources and provides abstractions like processes, threads, virtual memory, synchronization, and system calls so programs can run concurrently, safely, and efficiently.